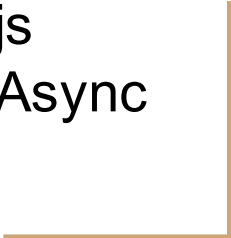




Web Application Development II

Week 5: Vue.js
Event Handling & Async
Requests



Agenda

Event Handling

Lifecycle Hooks

Using JavaScript Expressions

Computed Property

Form Input Binding (extended)

List Rendering

Asynchronous requests with Vue (Axios)



Using Bootstrap with Vue 3

In our examples and exercises, we shall be using Bootstrap for styling.

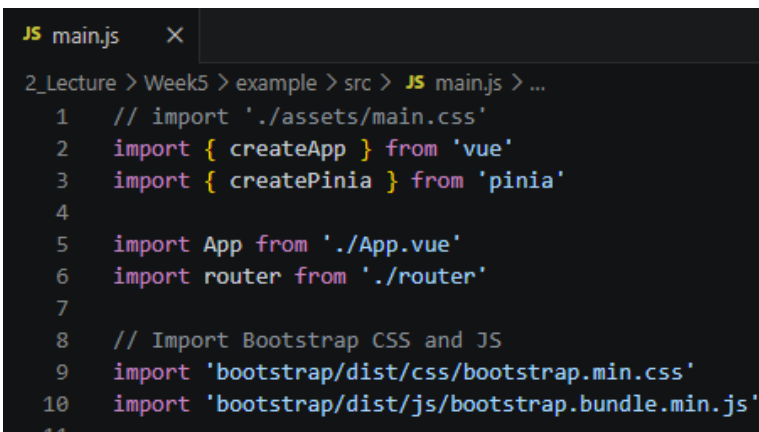
Before we can use Bootstrap in our Vue apps, we need to install Bootstrap first.

In the terminal, run the following command:
pnpm add bootstrap

```
C:\Users\Jackie Chan\Desktop\is216\2_Lecture\Week5\example>pnpm add bootstrap
✓ Lockfile passes supply-chain policies (verified 1d ago)
[WARN] 1 deprecated subdependencies found: glob@10.5.0
Already up to date
Progress: resolved 444, reused 400, downloaded 0, added 0, done
Done in 2.2s using pnpm v11.9.0
```

Observe in **main.js** of the given Vue apps example, resource folders, bootstrap libraries are imported as follows:

```
import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap/dist/js/bootstrap.bundle.min.js'
```



```
JS main.js  X
2_Lecture > Week5 > example > src > JS main.js > ...
1  // import './assets/main.css'
2  import { createApp } from 'vue'
3  import { createPinia } from 'pinia'
4
5  import App from './App.vue'
6  import router from './router'
7
8  // Import Bootstrap CSS and JS
9  import 'bootstrap/dist/css/bootstrap.min.css'
10 import 'bootstrap/dist/js/bootstrap.bundle.min.js'
```

Event Handling (1)

Use v-on directive to listen to DOM events and run some JavaScript when they're triggered

DOM Events such as

- Clicking a mouse
- Hovering over an image
- Typing something in a search bar

Example: *Example1.vue*

Syntax

```
<div v-on:eventname="expr"></div>
```

where eventname is any standard DOM event name (e.g. click, submit, change, focus, keyup and onEnter) and “expr” is a Javascript expression or function name.

“expr” is typically a method in the vue instance.

Event Handling (2)

When listening for keyboard events, we often need to check for specific keys. Vue allows adding key modifiers for v-on when listening for key events.

Commonly-used key codes: .enter, .tab, .delete (for both “Delete” and “Backspace”), .esc, .space, .up, .down, .left, and .right.

Example: *Example1.vue*

```
<!-- invoke hello() when `Enter` key is entered -->  
<input v-on:keyup.enter="hello(name)">
```

Events

Events reference: [Event reference | MDN](#)

Event	Description
click	The user clicks an HTML element
change	An HTML element has been changed
input	The user inputs something in an HTML element
mouseover	The user moves the mouse over an HTML element
mouseout	The user moves the mouse away from an HTML element
keypress	The user presses a key on the keyboard
load	The browser has finished loading the page

Exercise 1: Event Handling

In Ex1.vue, add code so that

- the shape changes to square on mouse over event
- And it changes back to circle on mouse out event.



Lifecycle Hooks

Lifecycle hooks are functions you run at specific stages of a component's life:

created (setup starts)

mounted (DOM rendered)

updated (reactive state changes → re-render)

unmounted (component removed)

They help you control *when* to load data, connect/disconnect resources, or interact with the DOM.

In this course, we shall only focus on `onMounted`

Example: *Example13.vue*

`onCreated()` `=> { ... }`

Runs **after** the component is created. **Use for:** initial API calls, initializing data

`onMounted()` `=> { ... }`

Runs **after** the component is inserted into the DOM. **Use for:** initial API calls, DOM access (focus input), initializing data

`onUnmounted()` `=> { ... }`

Runs **before/when** the component is removed from the page. **Use for:** cleanup (clearInterval, remove event listeners, close sockets)

Using JavaScript Expressions (1)

Not only binding simple property like name in templates, **Example:** *Example2.vue*
we can also use JavaScript expressions

```
<template>
  {{ number + 1 }}
  {{ ok ? 'YES' : 'NO' }}
  {{ message.split('').reverse().join('') }}

  <div :id="`list-${id}`"></div>
</template>
```

Using JavaScript Expressions (2)

Expressions Only

Each binding can only contain **one single expression**.

An expression is a piece of code that can be evaluated to a value. A simple check is whether it can be used after return.

Example: This does not work! Try it yourself!

```
<template>
  <!-- this is a statement, not an expression: -->
  {{ var a = 1 }}

  <!-- flow control won't work either, use ternary
  expressions -->
  {{ if (ok) { return message } }}</template>
```

Exercise 2: JavaScript Expressions

In Ex2.vue, you are building a tiny scoreboard game between two teams. The UI should:

1. Show team names and current scores.
2. Let the user click buttons to increase either team's score.
3. Automatically compute and display useful information (total points, points left to win).
4. Show a “winner” message when a team reaches the maximum score.

You will **NOT** use computed properties for this activity. Instead, you will practice writing small JavaScript expressions **directly inside the template** using `{{ }}`.

Continue next page ...

Mini Scoreboard

Falcons vs Tigers

Current: 0 - 0

Total points: 0

Points left to win: 10

+ Team A

+ Team B

Reset

No winner yet. Keep playing!

Exercise 2: JavaScript Expressions

Requirements

Part A: Add code in

`<!-- A. Event handlers go here -->`

so that when the user clicks the “+ Team A”,
“+ Team B”, “Reset” buttons,
“addA”, “addB”, and “reset” methods are
invoked accordingly.

Part B: Add code in

`<!-- B. In-template expressions go here -->`

So that the total points and points left to win
are shown when the user clicks the the “+
Team A”, “+ Team B”, “Reset” buttons

Mini Scoreboard

Falcons vs Tigers

Current: 1 - 0

Total points: 1

Points left to win: 9

No winner yet. Keep playing!

Continue next page ...

Exercise 2: JavaScript Expressions

Requirements

Part C: Add code in

`<!-- C. Display winner / status here -->`

so that when a team reaches 10 points, it
shows the winner status

Mini Scoreboard

Falcons vs Tigers

Current: 10 - 9

Total points: 19

Points left to win: 0

+ Team A

+ Team B

Reset

Winner: FALCONS

Computed Property

Some properties may be derived from (dependent on) other (native) properties.

In this case, it is better to use computed properties.

A computed property is a function which is used as if it is a variable.

Example: *Example3.vue*

```
<script setup>
  import { ref, computed } from 'vue'

  const fName = ref('Jack')
  const lName = ref('Daniel')

  const fullName = computed( () => {
    return fName.value + ' ' + lName.value
  })
</script>

<template>
  <p> {{ fullName }} </p>
</template>
```

Computed vs Methods

Computed

A computed property can be achieved by invoking a method.

Computed properties are cached based on their reactive dependencies.

A computed property only re-evaluates when some of its reactive dependencies have changed.

Methods

A method invocation always runs the function whenever a re-render happens.

Example: *Example3.vue*

```
function getFullName() { // method
  return fName.value + ' ' + lName.value
}
```

===

```
<p>Full Name: "{{ getFullName() }}"</p>
```

Form Input Binding: checkbox

Use v-model to create **two-way data bindings** on form input, textarea, and select elements.

Example: *Example4.vue*

```
<script setup>
  import { ref } from 'vue'

  const selectedFruits = ref([]) // can also set an initial value like ['Apple', 'Banana']

</script>

<template>
  <input type="checkbox" name="fruits" value="Apple" v-model="selectedFruits"> Apple
  <input type="checkbox" name="fruits" value="Orange" v-model="selectedFruits"> Orange
  <input type="checkbox" name="fruits" value="Banana" v-model="selectedFruits"> Banana

  <p>You selected {{ selectedFruits }} </p>
</template>
```

☑ Apple ☑ Orange ☑ Banana

You selected ["Apple", "Banana", "Orange"]

Form Input Binding: radio

Use v-model to create **two-way data bindings** on form input, textarea, and select elements.

Example: *Example4.vue*

```
<script setup>
  import { ref } from 'vue'

  const picked = ref("") // can also set an initial value like 'two'
</script>

<template>
  <input type="radio" name="num" v-model="picked"> None <br>
  <input type="radio" name="num" value="One" v-model="picked"> One <br>
  <input type="radio" name="num" value="Two" v-model="picked"> Two <br>

  <span>You picked: {{ picked }}</span>
</template>
```

☐ None
☐ One
☒ Two
You picked: Two

Form Input Binding: select

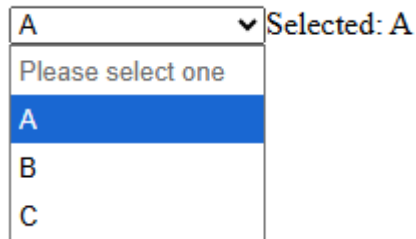
Use v-model to create **two-way data bindings** on form input, textarea, and select elements.

Example: *Example4.vue*

```
<script setup>
  import { ref } from 'vue'

  const selected = ref("") // can also set an initial value like 'B'
</script>
```

```
<template>
  <select v-model="selected">
    <option disabled value="">Please select one</option>
    <option>A</option>
    <option>B</option>
    <option>C</option>
  </select>
  <span>Selected: {{ selected }}</span>
</template>
```



A

Please select one

A

B

C

Selected: A

Binding to Dynamic Values

For radio, checkbox and select options, the v-model binding values are usually static strings (or booleans for checkbox).

Sometimes we may want to bind the value to a dynamic value.

We can combine **v-model** with **v-bind** to achieve that:

Example: *Example5.vue*

```
<script setup>
  import { ref } from 'vue'

  const pick = ref("")
  const a = ref('AAA')
  const b = ref('BBB')
</script>

<template>
  <input type="radio" v-model="pick" :value="a">A
  <input type="radio" v-model="pick" :value="b">B
  <p> Picked: {{ pick }} </p>
</template>
```

Binding to Computed Property

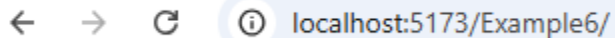
We can also bind to a computed property

Example: *Example6.vue*

```
const isActive = ref(true)
const error = ref(null)
const classObject = computed(() => ({
  active: isActive.value && !error.value,
  'text-danger': error.value
}))
```

====

```
<div :class="classObject">Example</div>
```



← → ↻ ⓘ localhost:5173/Example6/

Example 6

v-model Modifiers

v-model can be associated with different modifiers which helps to pre-process the inputs.

Example:

```
<!-- user input to be automatically typecast as a Number -->  
<input v-model.number="age">
```

```
<!-- whitespace from user input to be trimmed -->  
<input v-model.trim="msg">
```

v-model vs v-bind

v-bind

v-bind is used mainly for binding HTML attributes, e.g., src, class, style, and value.

Example

```
<a v-bind:href="url"> ... </a>
```

v-model

v-model is used mainly for input and form binding; use it when dealing with various input types. It is a two-way binding for form inputs.

Example

```
<input v-model="something">
```

// is the same as

```
<input v-bind:value="something"  
  v-on:input="something = event.target.value" >
```

Exercise 3

Given Ex3.vue, complete the TODO so that it produces a web page that accepts three inputs: x and an operator (+ - * / or %) and y, and display the result accordingly.

Hint: Use computed property.

x

y

= 12

x

y

= 20

List Rendering with v-for (1)

Use v-for directive to render a list of items based on an array.

The v-for directive supports a special syntax in the form of “item in items” (or “item of items”), where items is the source data array and item is an alias for the array element being iterated on.

v-for also supports an optional second argument for the index of the current item.

Example: *Example7.vue*

```
const items1 = ref(['a', 'b', 'c'])
```

====

```
<ul id="demo1">
  <li v-for="(item, idx) in items1">
    {{ idx }} - {{ item }}
  </li>
</ul>
```

List Rendering with v-for (2)

v-for can also take an integer, in which case it will repeat the template that many times.

Example: *Example8.vue*

```
const range = ref(10)
```

```
=====
```

```
<div id="demo1">  
  <span v-for="n in range">  
    {{ n }} &nbsp;  
  </span>  
</div>
```

List Rendering with v-for (3)

v-for can also render objects.

Order is important. The first parameter should be the value, followed by name and index.

Example: *Example9.vue*

```
const object = ref({  
  title: 'Foo',  
  author: 'Bar',  
  isbn: '12236'  
})
```

====

```
<ul id="demo3">  
  <li v-for="(value, name, index) in object">  
    {{ index }}. {{ name }}: {{ value }}  
  </li>  
</ul>
```

List Rendering with v-for (4)

We can also use a `<template>` tag with `v-for` to render a block of multiple elements.

Example

```
<ul>
  <template v-for="item in items">
    <li>{{ item.msg }}</li>
    <li class="divider">=====</li>
  </template>
</ul>
```

List Rendering with v-for (5)

Note: When v-if and v-for are used together in the same HTML node, v-if has a higher priority than v-for. Not recommended to use them together. See below example:

```
<!-- This will throw an error because v-if is evaluated first  
and property "student" is not defined yet -->
```

```
<li v-for="student in students" v-if="student.isInDeanList"> {{ student.name }} </li>
```

```
<!-- This can be fixed by moving v-for to a wrapping <template> tag -->
```

```
<template v-for="student in students">  
  <li v-if="student.isInDeanList">  
    {{ student.name }}  
  </li>  
</template>
```

Rendering a Table

v-for can be used to render tables easily.

Example: *Example10.vue*

Take note of the clear separation of data and presentation.

Exercise 4

Given Ex4.vue, complete the TODO so that it implements the following functionalities.

- When clicking the 'Delete' button, the corresponding item is removed from the list.
- When 'Add!' button is clicked, the item entered in the text box is added to the list.

Shopping Cart

- keyboard
- mouse
- iPhone
- macbook
- adapter



Shopping Cart

- keyboard
- mouse
- iPhone
- macbook
- adapter



Shopping Cart

- keyboard
- mouse
- iPhone
- macbook
- adapter
- Candy

Understanding Callbacks, Promises, and Async/Await (Recap from IS113)

A Promise is an object that represents the eventual success or failure of an asynchronous operation. When using a promise-based API like **fetch()**, the operation begins immediately and returns a Promise. Handlers can be attached to this Promise to execute once the operation either succeeds or fails.

A Promise has three possible states:

- Pending: The operation is still in progress.
- Fulfilled: The operation completed successfully, and the `.then()` handler runs.
- Rejected: The operation failed, and the `.catch()` handler runs.

Asynchronous Requests (1)

Asynchronous requests are **web requests that start now but finish later**, without “freezing” the rest of your program/UI while waiting for the response (e.g., from an API, database service, or file).

Why asynchronous requests matter: they let you

- Keep the UI responsive while loading data (no “page freeze”).
- Fetch data (JSON, Text, XML) from backend server/API or send data to backend.
- Handle multiple requests concurrently (load user + products + messages at once).

Vue itself doesn't contain any library to make Async. requests

However, it can work with existing, external libraries such as Axios for making Async. requests.

Axios is similar to Fetch used in WAD1, but we are using Axios in WAD2 because it is the recommended library for Vue.

Asynchronous Requests (2)

To use Axios library, we need to install the package first.

In the terminal, run

`pnpm add axios`

```
C:\Users\Jackie Chan\Desktop\is216\2_Lecture\Week5\example>pnpm add axios
✓ Lockfile passes supply-chain policies (verified 12m ago)
[WARN] 1 deprecated subdependencies found: glob@10.5.0
Already up to date
Progress: resolved 442, reused 398, downloaded 0, added 0, done
Done in 2.1s using pnpm v11.9.0
```

Axios

Promise based API for sending Asynchronous HTTP requests to backend server / REST endpoints and doing create/read/update/delete (CRUD) operations

Promise is an object used as a proxy for a value not yet known. **Recall that Promise is covered in WAD1**

Basic Axios Methods:

```
axios.get(url[, config])  
axios.post(url[, data[, config]])  
axios.put(url[, data[, config]])  
axios.delete(url[, data[, config]])
```

Axios Response Object

After sending an async. request, server returns a response.

Axios response object consists of:

data / payload: text, JSON, XML, etc.

Status: HTTP code (2xx, 3xx, 4xx)

statusText: HTTP status message ('OK', 'Moved Permanently', 'Client Error')

headers: information about the response, e.g. content type, cookies, etc.

(non-examinable) then() and catch()

Given a Promise object x,

x.then(fn) executes the function fn if the x is in a fulfilled state.

x.catch(fn) execute the function fn if x is in a rejected state.

Example

```
//returns a promise
axios.get('http://api.advice Slip.com/advice')
//call then to handle the good case
.then(response => {
    console.log(response.data.slip.advice)
})
//call catch to handle the bad case
.catch(error => {
    console.log(error.message)
})
```

(non-examinable) .then .catch

The asynchronous, promise-based API **axios.get** returns a Promise object.

A handler function is attached to the **.then()** method and runs when the Promise is in the "fulfilled" state."

```
const fetchPromise = await axios.get("https://api.adviceslip.com/advice");  
  
fetchPromise  
  .then((response) => {  
    console.log(`Received response: ${response.status}`);  
  })  
  .catch((error) => {  
    console.error(`Fetch error: ${error}`);  
  });
```

Console.log('apple')

A handler function is attached to the **.catch()** method and runs when the Promise is in the "Rejected" state."

Promise Code
Example

async/await

async/await provide a more synchronous-looking syntax for handling asynchronous operations, making code more readable and easier to manage compared to traditional callback functions or explicit Promise chaining with then/catch

The async keyword is used to declare that a function will always return a Promise (implicitly). The await keyword can only be used in an async function. When await is placed before an expression that returns a Promise, it pauses the execution of the async function until that Promise settles (either resolves or rejects).

In the example on the right, asyncCall is an asynchronous function. The await keyword is used to pause execution of asyncCall until axios.get (or any async function) completes, allowing for a more sequential-looking flow.

Error handling is managed with a standard try...catch block.

Example: *Example11.vue*

```
async function asyncCall() {  
  try {  
    const response = await axios.get(url)  
  
    const data = response.data  
    console.log(data)  
  } catch(error) {  
    console.error(error.message)  
  }  
}
```

remember to run this in terminal before running *Example11.vue*:

pnpm add axios

Axios: GET method

Send **GET** requests with parameters

Example: *Example12.vue*

```
try {  
  const response = await axios.get(url, {  
    params: {  
      name1: value1,  
      name2: value2  
    },  
  })  
  console.log(response.data)  
} catch (error) {  
  console.log(error.message)  
}  
....
```

Axios: POST method

Send **POST** requests with parameters

- Passing plain JavaScript object to POST request body

Example *Exercise 5*

```
try {  
  const response = await axios.post(url, {  
    name1 : value1,  
    name2 : value2  
  })  
  console.log(response.data)  
} catch (error) {  
  console.log(error.message)  
}  
....
```

Axios: PUT method

send an **HTTP PUT** request, with parameters, to a server, typically to completely replace an existing resource or create it if it does not exist

Example: *Example14.vue*

```
try {  
  const response = await axios.put(url, {  
    name1 : value1,  
    name2 : value2  
  })  
  console.log(response.data)  
  
} catch (error) {  
  console.log('There was an error: ' + error.message)  
}
```

Axios: DELETE method

send HTTP DELETE requests to a server to remove a specific resource.

Unlike POST or PUT requests, a DELETE request typically passes the resource identifier directly within the URL string or via parameters rather than an explicit body payload.

Example: *Example15.vue*

```
try {  
  
  // Payload must be inside the 'data' property  
  const response = await axios.delete(url,  
    { data: { subject: subject } })  
  
  console.log(response.data)  
  
} catch(error) {  
  console.log('There was an error: ' + error.message)  
}
```

Axios: Request Headers

In Axios methods, we can also configure and send **request headers** such as authorization token via config parameter:

```
axios.post(url, data, config)
```

Basic Axios Methods:

```
axios.get(url[, config])  
axios.post(url[, data[, config]])  
axios.put(url[, data[, config]])  
axios.delete(url[, data[, config]])
```

The server side (express.js) can access request headers using request.header, e.g., request.header("Authorization")

Example:

```
const url = 'https://example.com';  
  
const data = {  
  title: 'Hello World',  
  body: 'This is my post content.'  
}  
  
const config = { headers:  
  { 'Authorization': 'Bearer YOUR_ACCESS_TOKEN',  
    'Content-Type': 'application/json'  
  }  
}  
  
try {  
  const response = await axios.post(url, data, config)  
  ...
```

Running example: A Blog Service (1)

Set up a blog backend service using **Node.js + Express.js**

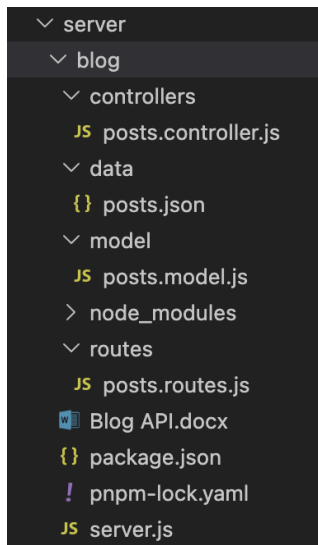
These technologies are covered in WAD1.

1. Download **server.zip** and unzip into a folder, say **server**. It contains a folder **'blog'** which is a blog backend app.
2. In the terminal, run the following commands to launch this backend service

```
cd server/blog
```

```
pnpm install
```

```
pnpm dev
```



```
(base) smu@smus-mbp Week5 % cd server
(base) smu@smus-mbp server % cd blog
(base) smu@smus-mbp blog % pnpm dev
Packages: +99
Progress: resolved 99, reused 99, downloaded 0, added 99, done

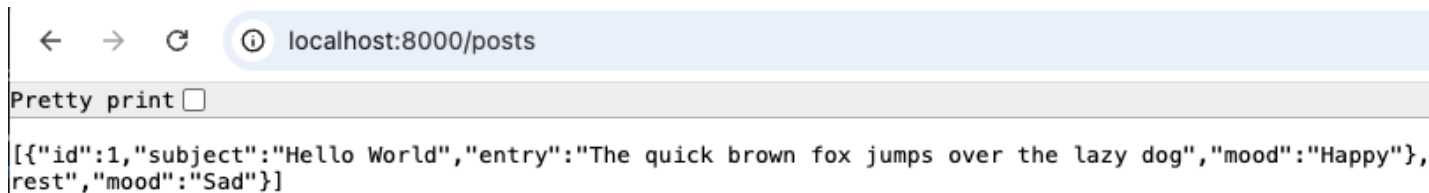
dependencies:
+ cors 2.8.6
+ express.js 1.0.0
+ nodemon 3.1.14

Done in 430ms using pnpm v11.15.1
$ nodemon server.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
API running on http://localhost:8000
```

Running example: A Blog Service (2)

3. In the browser, run the URL: <http://localhost:8000/posts>

You should observe the JSON data returned by the blog service.



```
localhost:8000/posts  
Pretty print ☐  
[{"id":1,"subject":"Hello World","entry":"The quick brown fox jumps over the lazy dog","mood":"Happy"},  
rest","mood":"Sad"}]
```

4. This blog service provides a few APIs. Some of them are:

- retrieve all posts: GET /posts

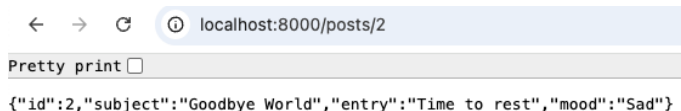
- retrieve post with a given id: GET /posts/:id

- create a new post: POST /posts

- update an existing post with a given id: PUT /posts/:id

- delete an existing post with a given id: DELETE /posts/:id

Example *retrieve post with a given id =>*



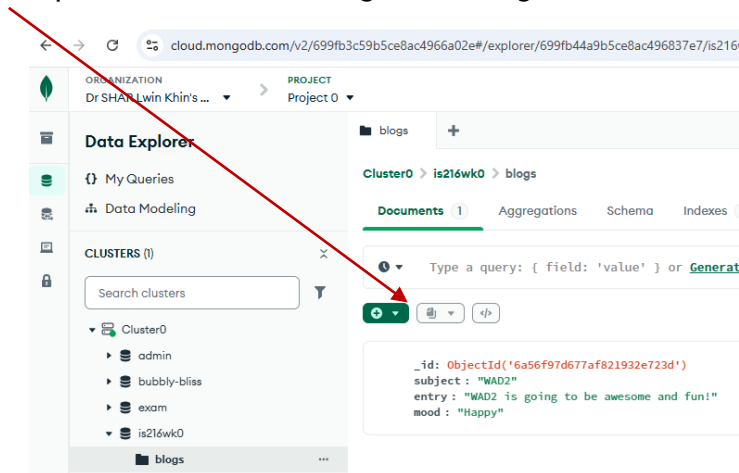
```
localhost:8000/posts/2  
Pretty print ☐  
{"id":2,"subject":"Goodbye World","entry":"Time to rest","mood":"Sad"}
```

Alternative: A Blog Service with MongoDB (1)

Set up a blog backend service using **Node.js + Express.js + Mongoose + MongoDB**

These technologies are covered in WAD1.

1. Download **server.zip** and unzip into a folder, say *server*. It contains a folder '**blog-with-mongodb**' which is a blog backend app.
2. Setup [MongoDB account](#)
3. Import `blogs-data.json` provided in *server/blog-with-mongodb/data*



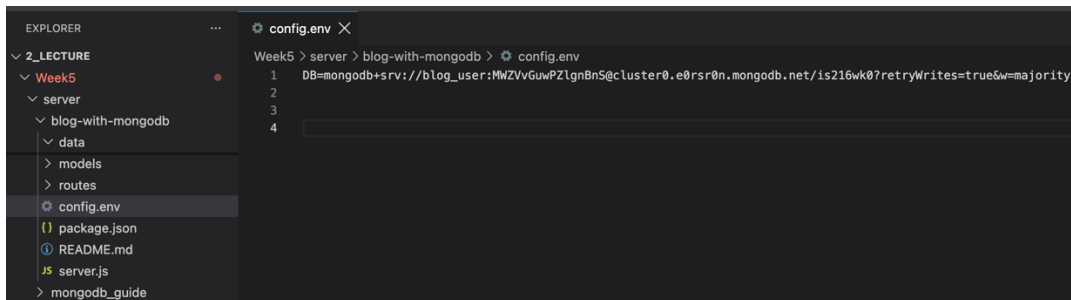
Instead of using the blog service set up in previous slides, you could alternatively use a more realistic backend blog service with MongoDB.

You could use this as a reference for your project.

(Refer to **mongodb_guide.pptx** given in *REST/mongodb/* folder to conduct the following steps)

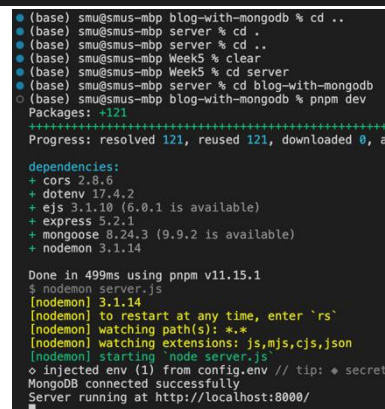
Alternative: A Blog Service with MongoDB (2)

- Get the database connection string and replace the placeholder in config.env with your database connection string, e.g.,
 DB=mongodb+srv://blog_user:MWZVvGuwPZlgnBnS@cluster0.e0rsr0n.mongodb.net/is216wk0?retryWrites=true&w=majority



The screenshot shows the VS Code Explorer on the left with the file tree expanded to 'server > blog-with-mongodb > config.env'. The main editor displays the contents of 'config.env', which contains a single line of MongoDB connection string: `DB=mongodb+srv://blog_user:MWZVvGuwPZlgnBnS@cluster0.e0rsr0n.mongodb.net/is216wk0?retryWrites=true&w=majority`.

- In the terminal, run the following command to install the packages and launch the web application
 cd server/blog-with-mongodb
 pnpm dev



The screenshot shows the terminal output for the command `pnpm dev`. It displays the progress of installing dependencies, including a list of installed packages like `cors`, `dotenv`, `ejs`, `express`, `mongoose`, and `nodemon`. The output concludes with the message: `Server running at http://localhost:8000/`.

Running example: A Blog Service (3)

blog web application provides 5 API services

Example: *Example13.vue*

- retrieve all posts: GET /posts

- retrieve post with a given id: GET /posts/:id

- create a new post: POST /posts

- update an existing post with a given id: PUT /posts/:id

- delete an existing post with a given id: DELETE /posts/:id

We can send Asynchronous Requests to this blog application via
Vue and Axios.

Test Case: <http://localhost:8080/posts>

Run this URL in your browser and check that some JSON data is
received

Exercise 5

Setup the server as instructed “Running example: A Blog Service” slides

Complete the TODO in Ex5.vue so that it uses the POST API (<http://localhost:8000/posts>) provided by the Blog Service (running example) to add a new blog post.

Add a New Blog Post

Subject:

Entry:

VueJS is awesome!!!

Mood:

[Click here to return to Main Page](#)

Optional Utility Tool: Postman

Postman is a tool for **testing and debugging (REST) APIs**.

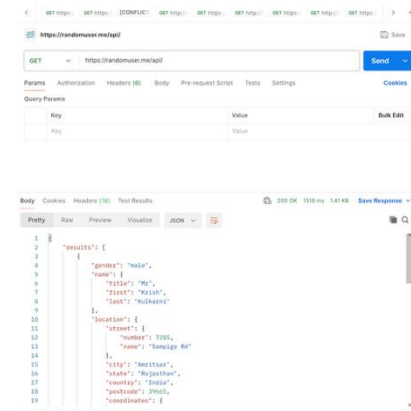
It lets you send HTTP requests (e.g., **GET, POST, PUT, DELETE**) and inspect:

response **status code** (200, 404, 500...)

response **body** (JSON)

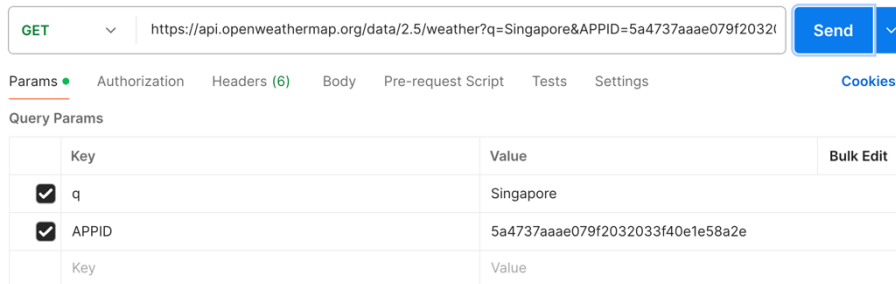
response **headers**

request **time/latency**



<https://api.openweathermap.org/data/2.5/weather?q=Singapore&APPID=5a4737aaae079f2032033f40e1e5...>

Save



You can use Postman for:

- Testing endpoints w/o building a UI first
- Quickly verify API behavior: **CRUD**, JSON data, validation, errors



Take Away Message

Use “computed” whenever possible.

Take note of all inputs are stored with v-model.

Use v-for to render structured data.

JSON - a structured text for server-client communication

Axios – send asynchronous HTTP GET/POST requests